USING MOSTEK'S F8 IN A SCANNED KEYBOARD APPLICATION

Using Mostek's F8 In A Scanned Keyboard Application

BLOCK DIAGRAM OF
4x4 KEYBOARD MATRIX

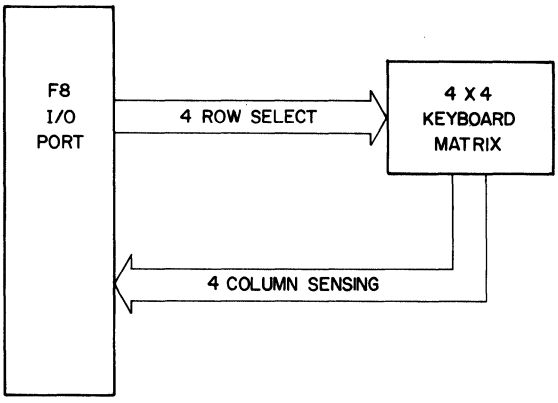


Figure 1

INTRODUCTION

Many microprocessor based systems require input from a keyboard of some type. The hardware required to encode a keyboard outside of the processor can be eliminated by using a keyboard scanning technique. With one F8 port, a 16 switch keyboard can be scanned (see fig. 1) using no external hardware. This is because of the bi-directional quality of the F8 ports.

F8 Family

THEORY OF OPERATION

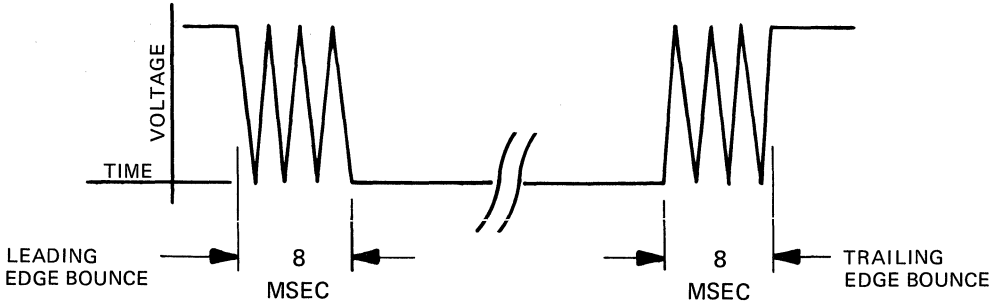
When scanning the keyboard, one of the four row select bits is turned on supplying a ground return for one row of switches. The column data is then read back into the processor via the four column bits. These four bits will indicate the condition of all four switches in the selected row. Each of the four rows is selected, one at a time, continously providing current status of all 16 switches.

"BOUNCE" is a problem encountered when using mechanical switches (see fig. 2). In order to prevent multiple detection of the switch closure, the bounce must be filtered out. A conventional solution to the bounce problem was to use an R—C filter and attempt to eliminate it electrically. However, when using the F8 scanning technique the switch bounce can be filtered in software by taking multiple samples of the switch to verify switch depression and release.

Since the software must usually scan all switches continuously, a register (or half) can be used to maintain the status of each switch.

A common requirement for keyboards is "N—key rollover", meaning that if more than one switch is depressed at a time, all switch closures will be detected. This requirement can be met when using the scanning technique as described above. Since all switches are continuously scanned, the condition of each switch is always available to the processor.

SWITCH BOUNCE



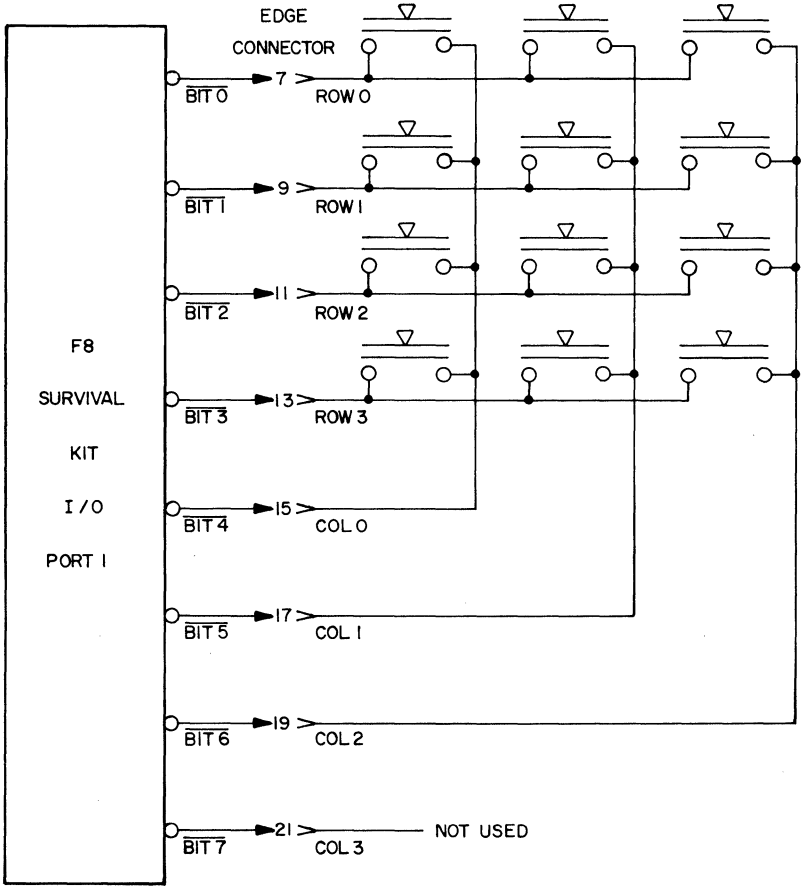


Figure 3

EXAMPLE HARDWARE DESIGN

The example in figure 3 shows a 4x3 matrix interfaced to an F8 port. This arrangement will provide N–key rollover input to the processor unless three keys are depressed simultaneously to form an L configuration. Then erroneous input could occur. If this presents a problem for a given application, one germanium diode (1N 270) should be added on the column pole of each switch (see fig. 4).

The operation of this keyboard (fig. 3) is simple. To sense the condition of row 0, a Hex '01' is written to port 1. Port 1 is then read back. The state of bits 4, 5 and 6 (COL 0, COL 1, COL 2) will be 1 if the respective switches in row 0 are closed and 0 if

FOR SOME APPLICATIONS DIODES ARE NECESSARY

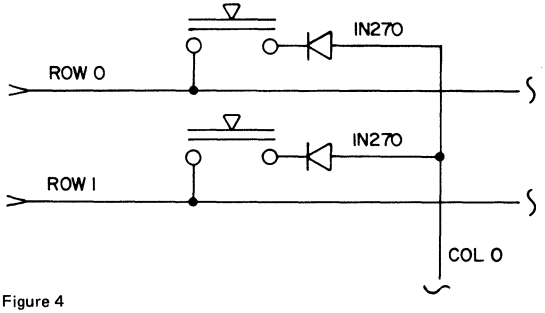


Figure 4

KEYBOARD SCAN ROUTINE (4 x 3 MATRIX)

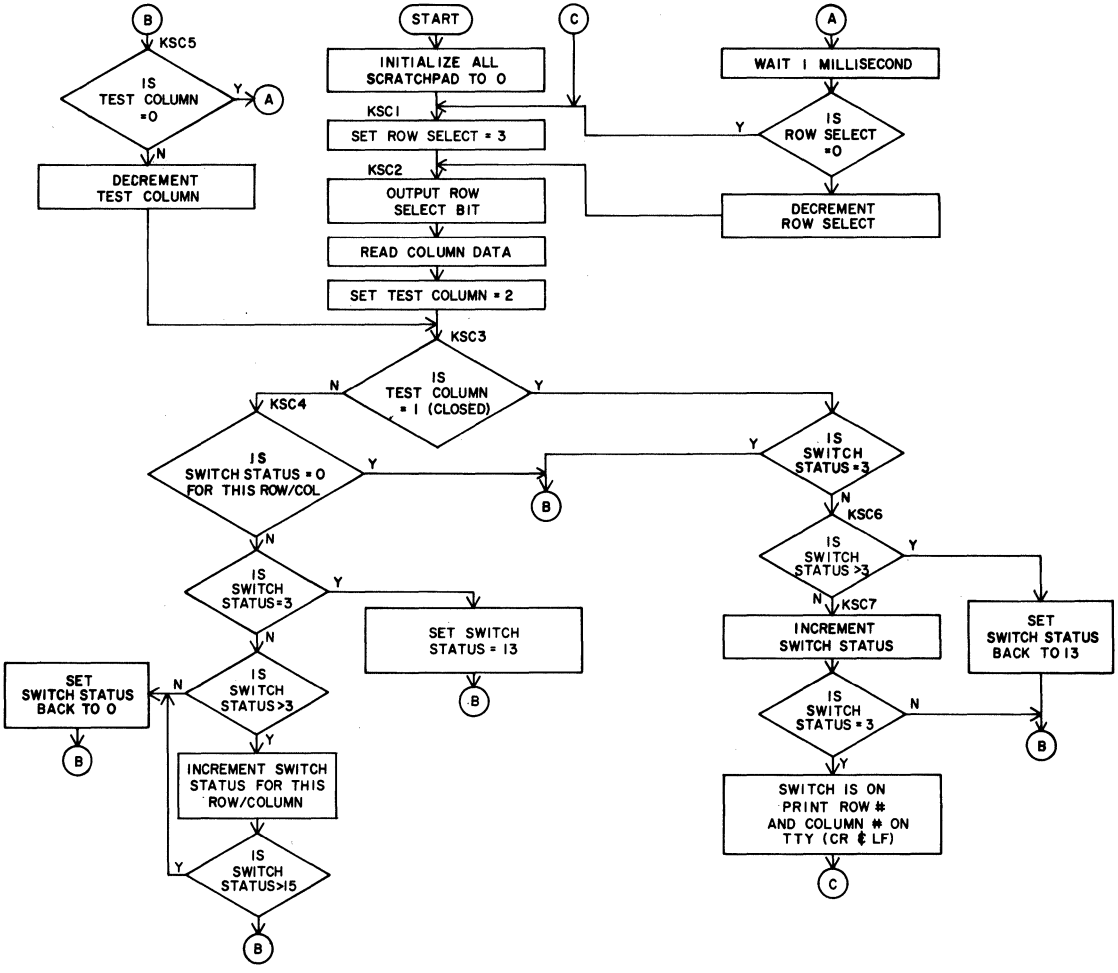


Figure 5

open. (Note: The F8 I/O ports contain internal pull-ups). The other three rows are read similarly.

EXAMPLE SOFTWARE FOR THE 4x3 MATRIX KEYBOARD

An example program was written to run on the F8 Survival Kit to demonstrate software switch sensing and debounce.

One scratchpad register is used to maintain current status for each switch. When a switch is inactive it maintains a status of 0. In order for the switch to be processed, three consecutive scans must occur in which the switch is sensed to be closed.

When a switch is first sensed closed, its status is incremented to 1. In succeeding scans its status is either incremented (if sensed closed) or reset to 0 (if sensed open) until the status reaches 3, thus requiring three consecutive scans with the switch closed.

The switch is then processed, which in the example means the column number and row number are printed on the TTY (terminal).

A status of 3 is maintained by the switch until the first time it is sensed open. At that time its status is set to 13. Then three consecutive scans with the switch open are required to get the switch back to inactive status (0). This is accomplished by incrementing the status (if sensed open) or resetting the status to 13 (if sensed closed) until it reaches 15. The status is then reset to 0. As long as bounce occurs, however, the status will be reset to 13.

The flowchart (fig. 5) shows the logic described above. Note that at the end of each row scan there is a one millisecond delay which effects an interscan delay of 4 milliseconds for each switch. This means that the switch must be on 'solid' for 8 milliseconds before being processed and off 'solid' 8 milliseconds before becoming inactive again; so the switch will only be processed one time per depression. This debounce time sets the max keyboard entry rate for a given switch at 1 entry/24 milliseconds.

Figure 6 shows the scratchpad register assignments used by the example program.

For an instruction by instruction description of the example program see the listing (fig. 7).

ALTERNATE DESIGN APPROACHES

When more than 16 switches are needed, an additional chip must be used. By adding a 4 to 16 decoder (see figure 8) to select 1 of 16 rows, up to 64 switches can be scanned.

Many off-the-shelf keyboards are available which have a 4x3 or 4x4 physical arrangement, but all switches have one common pole (on the P.C. Board). This type of keyboard can be scanned by using a 4 bit code to select one of up to 16 switches. The code is then decoded by a 4 to 16 decoder which supplies a ground return to the selected switch. The switch common line is then read to sense the condition of that switch (see figure 9).

If more ports can be assigned to the keyboard interface, other options may become advantageous. For example, with two ports 16 switches can be read without scanning. The basic requirements such as switch debounce and N-Key rollover will remain regardless of which option is taken. The best approach to a given design application will be determined by the system requirements and structure.

SCRATCHPAD REGISTER ASSIGNMENTS

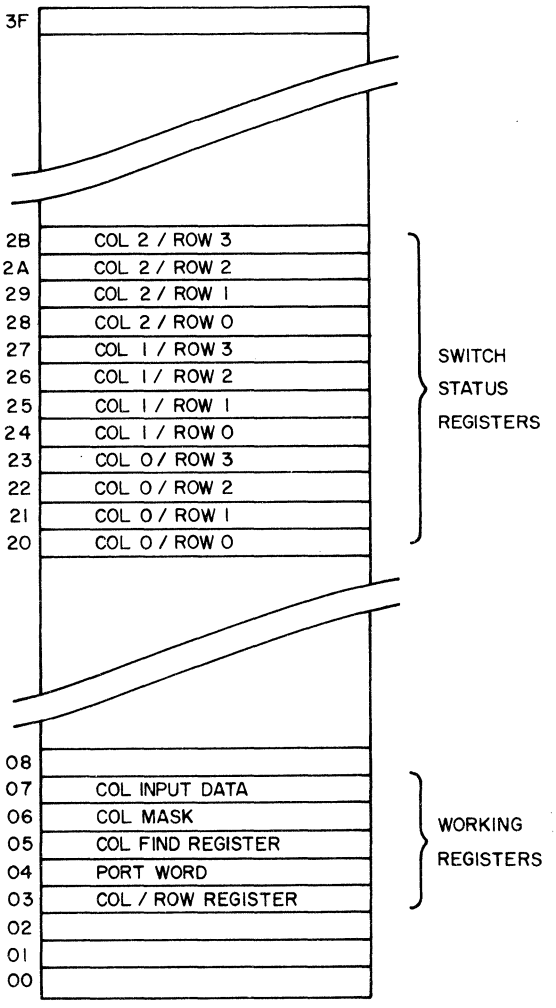


Figure 6

16 x 4 KEYBOARD

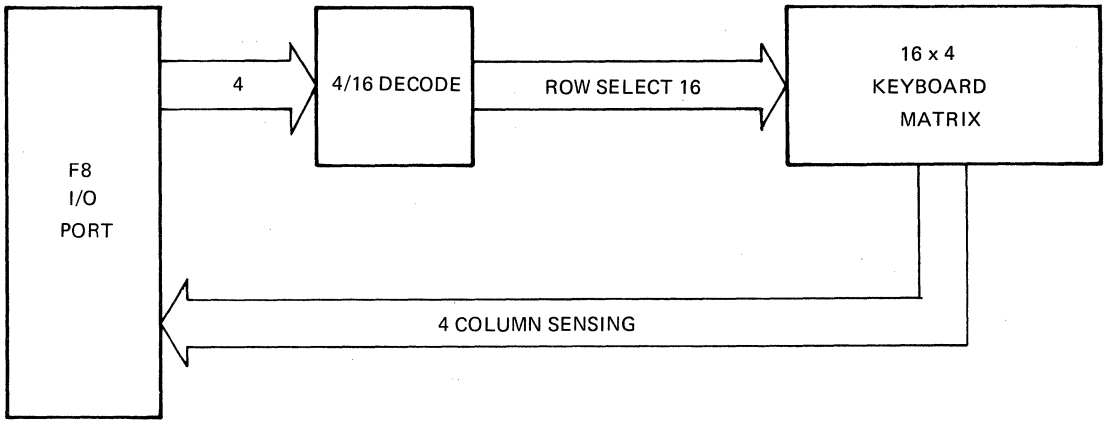


Figure 8

KEYBOARD WITH COMMON POLE

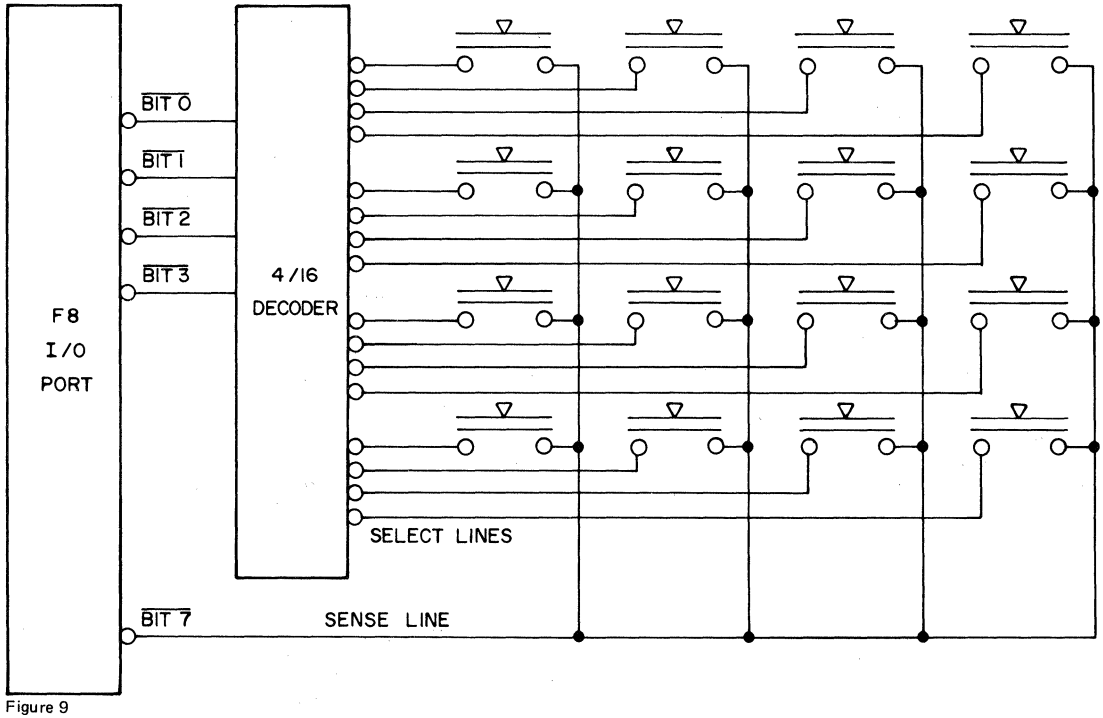


Figure 9